

WEEK 5 PRACTICAL

TASK 1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
GNU nano 7.2 producer_consumer.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    int pipefd[2];
    int data;
    pid_t pid;

    pipe(pipefd);

    pid = fork();

    if (pid > 0) {
        // Parent = Producer
        close(pipefd[0]);

        clock_t start = clock();

        for (int i = 1; i <= 5; i++) {
            write(pipefd[1], &i, sizeof(i));
            printf("Produced: %d\n", i);
        }

        close(pipefd[1]);
        wait(NULL);

        clock_t end = clock();

        printf("Time taken: %f seconds\n",
              (double)(end - start) / CLOCKS_PER_SEC);
    }
    else {
        // Child = Consumer
        close(pipefd[1]);

        while (read(pipefd[0], &data, sizeof(data)) > 0) {
            printf("Consumed: %d\n", data);
        }

        close(pipefd[0]);
    }

    return 0;
}
```

Output

```
varshini@DESKTOP-BLT5L3R:~$ nano producer_consumer.c
varshini@DESKTOP-BLT5L3R:~$ gcc producer_consumer.c -o producer_consumer
varshini@DESKTOP-BLT5L3R:~$ ./producer_consumer
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Time taken: 0.000178 seconds
varshini@DESKTOP-BLT5L3R:~$
```

Observation

The parent process acts as the **producer** and generates integer values from 1 to 5. These values are sent to the child process through an **anonymous pipe** using the `write()` system call. The child process acts as the **consumer** and receives the values using the `read()` system call. The communication time is measured to determine the efficiency of data transfer between the two processes.

Result

The producer-consumer communication system was successfully implemented using an anonymous pipe, and the communication time was measured successfully.

TASK 2

Develop a program that executes the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
GNU nano 7.2 pipe_ls.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];

    pipe(pipefd);

    if (fork() == 0) {
        // Child 1: ls -l
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
    }
    else {
        // Parent creates second child
        if (fork() == 0) {
            // Child 2: grep ".c"
            close(pipefd[1]);

            dup2(pipefd[0], STDIN_FILENO);
            close(pipefd[0]);

            execlp("grep", "grep", ".c", NULL);
        }

        close(pipefd[0]);
        close(pipefd[1]);

        wait(NULL);
        wait(NULL);
    }

    return 0;
}
```

Output

```
varshini@DESKTOP-BLT5L3R:~$ nano pipe_ls.c
varshini@DESKTOP-BLT5L3R:~$ gcc pipe_ls.c -o pipe_ls
varshini@DESKTOP-BLT5L3R:~$ ./pipe_ls
-rwxr-xr-x  1 varshini varshini 16640 Aug 29 06:21 escape
-rw-r--r--  1 varshini varshini  2016 Aug 29 06:20 escape.c
-rwxr-xr-x  1 varshini varshini 16400 Aug  3 06:04 fork_exec
-rw-r--r--  1 varshini varshini   946 Aug  3 06:04 fork_exec.c
-rw-r--r--  1 varshini varshini    81 Aug  5 14:20 hello.c
-rw-r--r--  1 varshini varshini   470 Aug  7 08:19 main.c
-rw-r--r--  1 varshini varshini  2755 Aug 29 06:05 myshell.c
-rw-r--r--  1 varshini varshini   694 Sep 18 03:59 pipe_ls.c
-rwxr-xr-x  1 varshini varshini 16312 Aug 29 06:36 process
-rw-r--r--  1 varshini varshini   810 Aug 29 06:33 process.c
-rwxr-xr-x  1 varshini varshini 16304 Aug 29 06:41 process_state
-rw-r--r--  1 varshini varshini  1246 Aug 29 06:41 process_state.c
-rwxr-xr-x  1 varshini varshini 16312 Sep 18 02:48 producer_consumer
-rw-r--r--  1 varshini varshini   839 Sep 18 02:48 producer_consumer.c
-rwxr-xr-x  1 varshini varshini    35 Aug  5 16:00 script.sh
lrwxrwxrwx  1 varshini varshini     5 Aug  5 16:09 shortcut -> file1
-rw-r--r--  1 varshini varshini   338 Aug 29 06:46 state.c
-rwxr-xr-x  1 varshini varshini 16312 Aug 30 11:43 wait_compare
-rw-r--r--  1 varshini varshini  1571 Aug 30 11:42 wait_compare.c
-rw-r--r--  1 varshini varshini   704 Aug 30 12:27 zombie.c
varshini@DESKTOP-BLT5L3R:~$ |
```

Observation

The program successfully implements the shell pipeline using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The output of `ls -l` is redirected through the pipe to `grep ".c"`, which displays the matching entries.

Result

The shell command `ls -l | grep ".c"` was successfully implemented using system calls.